

Frontend *to* backend, in 45 days.

A focused, day-by-day path from your first `<html>` tag to a deployed full-stack application — covering the markup, the logic, the framework, and the server-side glue that holds it all together.

45 DAYS	05 PHASES	10 TOOLS	01 CAPSTONE
------------	--------------	-------------	----------------



— COURSE DESIGNED BY

Ritik Verma — *Software Developer*

3+ years of experience building real-time solutions with HTML, CSS, JavaScript, React, Spring Boot, MySQL & Liferay.

01 Git VERSION CONTROL	02 Postman API TESTING	03 / 04 <i>HTML & CSS</i> MARKUP + STYLING	
05 JavaScript LOGIC	06 Bootstrap UI LIBRARY	07 React UI FRAMEWORK	08 Java LANGUAGE
09 Spring Boot BACKEND / API	10 MySQL DATABASE		

The *tools* of the trade.

Languages and frameworks get the spotlight, but the tools below are what you'll actually open every single day. Treat them as muscle memory, not afterthoughts.

— 01

Git

VERSION CONTROL

Track every change, branch fearlessly, and undo mistakes without panic. Learn init, add, commit, push, pull, branch, merge, and rebase.

— 03

Postman

API TESTING

Hit every endpoint before your frontend ever does. Build collections, save environments, attach JWT tokens, debug responses.

— 05

DevTools

BROWSER DEBUGGING

Chrome's Elements, Console, Network, and Application tabs. Inspect requests, debug JavaScript, profile performance, simulate devices.

— 07

MySQL

DATABASE

A relational database. Write SQL queries, design schemas with Workbench, and connect from Spring Boot via JDBC.

— 09

Maven

BUILD TOOL

Manage Java dependencies via pom.xml. Understand the build lifecycle — compile, test, package, install — and Spring starters.

— 02

GitHub

CODE HOSTING

Where your code lives publicly. Repos, README files, pull requests, issues, and GitHub Pages — your portfolio's permanent home.

— 04

VS Code

CODE EDITOR

Your daily workspace. Configure extensions, integrated terminal, Git integration, Prettier, ESLint, and live server.

— 06

npm

PACKAGE MANAGER

Install, update, and run JavaScript packages. Understand package.json, dependencies, scripts, and the node_modules folder.

— 08

IntelliJ

JAVA IDE

The serious Java IDE. Run Spring Boot apps, debug with breakpoints, refactor confidently, and use the database tool window.

— 10

Vercel

DEPLOYMENT

One-click frontend deploys. Pair with Render or Railway for backend hosting. Push to main, see it live in seconds.

Phase *01*

DAYS 01 – 09 • FRONTEND FOUNDATIONS

The *building blocks* every web page is made of.

Markup, styling, layout, a component library, and version control habits — by the end of phase 1 you'll have a deployed portfolio with a clean Git history.

DAY

01

Setup, Git basics & HTML fundamentals

Install VS Code, Git, and a browser. Configure Git with your name and email. Learn the anatomy of an HTML page — DOCTYPE, head, body, semantic tags, headings, paragraphs, lists, links, and images. Run your first `git init` and `git commit`.

DAY

02

HTML forms, tables & embedded media

Master input types, labels, select, textareas, validation attributes, table semantics, video/audio, and iframes. Build a registration form and a data table.

DAY

03

CSS basics — selectors, box model, color

Inline / internal / external CSS, selectors (class, id, attribute, pseudo), specificity, the box model, units (px, rem, em, %), and color systems.

DAY

04

Flexbox layouts

Flexbox for one-dimensional flow — flex-direction, justify-content, align-items, flex-wrap, gap. Build navbars, card rows, and centered hero sections.

DAY

05

CSS Grid & advanced layouts

CSS Grid for two-dimensional structure — grid-template-columns, grid-area, named lines. Build a magazine-style layout with grid as the primary technique.

DAY

06

Responsive design, transitions & DevTools

Mobile-first approach, viewport meta tag, breakpoints, fluid typography with `clamp()`, CSS transitions and keyframe animations. Master Chrome DevTools — Elements, device toolbar, Network tab.

DAY

07

Bootstrap 5 — grid system & components

Bootstrap CDN setup, the 12-column grid, containers, rows, breakpoints, and core components: buttons, cards, navbars, modals, forms, badges.

DAY

08

Bootstrap utilities & customization

Spacing, color, and display utilities. Customize Bootstrap with overrides, build a multi-section landing page, and use Bootstrap Icons. Practice the responsive breakpoint system.

DAY

09

GitHub, deployment & portfolio site

Combine HTML, CSS, and Bootstrap into a multi-page portfolio. Create a GitHub account, push your code with branches and pull requests, write a clean README, and deploy live via GitHub Pages or Netlify.

Phase 02

DAYS 10 – 18 • JAVASCRIPT DEEP DIVE

From static pages to *interactive* applications.

Vanilla JavaScript, end to end. By the end of phase 2 you'll be comfortable with closures, async patterns, and modern ES6+ — the prerequisites for React.

DAY

10

JavaScript core — variables, types, operators

let / const / var, primitive vs reference types, template literals, type coercion, comparison operators, conditionals (if/else, switch, ternary), and loops.

DAY

11

Functions, scope & closures

Function declarations vs expressions, arrow functions, parameters and rest/spread, lexical scope, hoisting, and closures — the foundation of advanced JavaScript patterns.

DAY

12

Arrays, objects & ES6+ features

Array methods (map, filter, reduce, find, forEach, sort), object literals, destructuring, shorthand syntax, spread/rest, optional chaining, and nullish coalescing.

DAY

13

DOM manipulation & events

querySelector, getElementById, modifying styles/attributes/innerHTML, creating and removing elements, event listeners, event bubbling, delegation, and form handling.

DAY

14

Async JavaScript — promises & async/await

The event loop, callbacks, callback hell, promises (.then/.catch/.finally), async/await syntax, error handling with try/catch, and Promise.all / Promise.race.

DAY

15

Fetch API & consuming REST endpoints

The Fetch API, GET / POST / PUT / DELETE requests, headers and CORS basics, JSON parsing, and consuming a public REST API like JSONPlaceholder or OpenWeather.

DAY

16

JavaScript modules, classes & OOP

ES6 modules (import/export), classes, constructors, inheritance with extends, static methods, and getters/setters. Refactor a script into clean modular code.

DAY

17

localStorage, error handling & debugging

Persist data with localStorage and sessionStorage, JSON.stringify / JSON.parse, robust try/catch patterns, and using Chrome DevTools Sources tab for breakpoint debugging.

DAY

18

Mini project — vanilla JS app

Build a complete app with vanilla JS — todo list, weather dashboard, or quiz. Use modules, async/await with a real API, localStorage persistence, and clean DOM updates. Push to GitHub.

Phase 03

DAYS 19 – 27 • REACT ECOSYSTEM

Components, hooks, and the *modern* React workflow.

Nine days to get genuinely fluent in React — not just toy components, but routing, global state, forms, and data fetching patterns you'll use on the job.

DAY

19

Node.js, npm, Vite & React essentials

Install Node.js and npm, understand package.json and node_modules, spin up a Vite project, master JSX syntax, build functional components, pass props, and grasp the virtual DOM.

DAY

20

useState & component composition

useState for local state, controlled forms, lifting state up between components, prop drilling, and composing reusable UI primitives like Button, Card, and Input.

DAY

21

useEffect, side effects & data fetching

useEffect for side effects, the dependency array, cleanup functions, fetching data with Axios in useEffect, and handling loading and error states cleanly.

DAY

22

Conditional rendering, lists & keys

Conditional rendering patterns (&&, ternary, early return), list rendering with map and stable keys, fragment shorthand, and avoiding common rendering bugs.

DAY

23

React Router & multi-page apps

react-router-dom v6, BrowserRouter, Routes / Route, nested routes, dynamic params with useParams, programmatic navigation with useNavigate, and 404 handling.

DAY

24

Forms, validation & custom hooks

Controlled vs uncontrolled forms, validation patterns, react-hook-form basics, and writing your own custom hooks like useForm and useFetch to extract reusable logic.

DAY

25

Context API & global state

React Context for theme and auth state, the Provider pattern, useContext, and when to reach for Context vs prop drilling vs a state library like Redux Toolkit or Zustand.

DAY

26

Performance, memoization & refs

useMemo, useCallback, React.memo, the useRef hook for DOM access and persistent values, lazy loading routes with React.lazy and Suspense, and avoiding unnecessary re-renders.

DAY

27

Mini project — React app with API

Build a feature-rich React app — task manager, movie browser, or notes app — with routing, Context for auth, custom hooks, real API integration, and Bootstrap styling. Deploy on Vercel.

Phase 04

DAYS 28 – 36 • BACKEND WITH JAVA & SPRING BOOT

Java, Spring Boot, MySQL — *everything* behind the API.

The heaviest phase. Layered architecture, JPA, validation, exception handling, JWT-secured endpoints, and basic testing — production patterns you'll see on day one of any backend job.

DAY

28

Java refresher & OOP fundamentals

Java syntax, primitive types, classes and objects, constructors, inheritance, polymorphism, abstraction, encapsulation, interfaces, collections (List, Map, Set), and exception handling.

DAY

29

Spring Boot setup, Maven & IntelliJ

Install JDK 17+, generate a project on start.spring.io, understand Maven (pom.xml, dependencies, lifecycle), the `@SpringBootApplication` entry point, and the layered architecture pattern.

DAY

30

REST APIs with Spring Web & Postman

`@RestController`, `@GetMapping` / `@PostMapping` / `@PutMapping` / `@DeleteMapping`, `@PathVariable`, `@RequestBody`, `ResponseEntity`. Install Postman, build collections, and test every endpoint thoroughly.

DAY

31

SQL fundamentals & MySQL Workbench

Install MySQL Server and Workbench, write your first SQL — CREATE, INSERT, SELECT, UPDATE, DELETE, JOIN, GROUP BY, primary and foreign keys. Design a schema visually.

DAY

32

Spring Data JPA & entity mapping

Connect Spring Boot to MySQL via JDBC, define `@Entity` classes with `@Id`, `@GeneratedValue`, `@Column`, create `JpaRepository` interfaces, and run real CRUD operations.

DAY

33

Service layer, DTOs & validation

Separate business logic into `@Service` classes, decouple API contracts with DTOs, use `ModelMapper` or `MapStruct`, and validate input with `@Valid` and Bean Validation annotations.

DAY

34

Exception handling & JPA relationships

Global `@ControllerAdvice` handlers, custom exceptions, consistent error responses, and JPA relationships — `@OneToMany`, `@ManyToOne`, `@ManyToMany` with cascade and fetch types.

DAY

35

Spring Security & JWT authentication

Configure Spring Security, BCrypt password encoding, login and register endpoints, generate and validate JWT tokens with `jjwt`, and protect routes with role-based access control.

DAY

36

Testing with JUnit & Mockito

Unit testing services with JUnit 5 and Mockito, mocking dependencies, integration testing controllers with `@SpringBootTest` and `MockMvc`, and basic test coverage with JaCoCo.

Phase 05

DAYS 37 — 45 • FULL STACK CAPSTONE

The *capstone*. Where everything clicks together.

Design, build, integrate, deploy, document, and present. Walk away from day 45 with a polished full-stack application on your GitHub — worth more than any certificate.

DAY

37

Project planning, schema & Git workflow

Pick a capstone (task manager, blog, e-commerce, booking app). Design the MySQL schema with Workbench, define endpoints, sketch wireframes, set up two GitHub repos with proper README, .gitignore, and a feature-branch workflow.

DAY

38

Backend build — entities & core APIs

Create JPA entities and relationships, repositories, service layer, and REST controllers for the core resources. Implement full CRUD and verify every endpoint with Postman.

DAY

39

Backend auth, validation & CORS

Wire up JWT-based login and registration, role-based authorization, request validation with @Valid, global exception handling, and configure CORS so the React dev server can talk to Spring Boot.

DAY

40

Frontend build — pages & routing

Login, register, dashboard, list view, detail view, profile — all with React Router. Style with Bootstrap and ensure mobile responsiveness across every page.

DAY

41

Connect frontend to backend with Axios

Create an Axios instance with a base URL, attach JWT tokens via interceptors, store tokens in localStorage, build protected routes with React Router, and wire up real API calls to every CRUD page.

DAY

42

State management & UX polish

Add Context for auth and theme, polish loading skeletons and error states, add toast notifications, implement search and filters, add pagination, and refine micro-interactions.

DAY

43

Testing, debugging & bug bash

End-to-end manual testing across browsers and screen sizes, fix every form validation message, write critical JUnit tests for services, basic React Testing Library tests, and serious DevTools work.

DAY

44

Deployment — frontend & backend live

Deploy the React build to Vercel or Netlify, package the Spring Boot app as a JAR and deploy to Render or Railway, host MySQL on a free tier (Aiven, Railway), wire up environment variables, and verify the live site works end-to-end.

DAY

45

Documentation, demo & resume polish

Write a clear README with setup steps, screenshots, architecture diagram, and the live link. Record a short demo video. Update GitHub and LinkedIn, add the project to your resume, and prepare to discuss it in interviews.

*Spend the first hours **learning**, the next hours **coding**, and the last hour pushing to GitHub. Consistency beats intensity — every single time.*

— Ritik Verma

45 DAYS · 5 PHASES · 1 DEPLOYED APP